

# Learning Notes: How the Single Actuator LSTM Simulation Works

Hussein Zolfaghari

May 27, 2026

## 1 Goal of This Document

This document explains, step by step, what the LSTM simulation does and why each part is needed. The goal is to make the method understandable even without prior background in LSTM networks. The MATLAB code trains a data driven dynamic model for one electromagnetic soft actuator. The model receives current as the input and predicts displacement and coil force as outputs.

## 2 What Problem Are We Solving?

In the physics based actuator network, the actuator array is modeled through mass spring damper equations and state space matrices. That approach explicitly defines the internal states and equations of motion. In the present dataset, the available data are for one actuator only. Therefore, instead of building a large state space network, the goal is to learn a single actuator input output relation directly from COMSOL data.

The model learns:

$$\text{current history} \longrightarrow \text{displacement and force history.} \quad (1)$$

This is not a static relation. It is a dynamic relation because the actuator output depends on previous current values, not only the current value at the same instant.

## 3 Why Not Use a Simple Curve Fit?

A simple curve fit might use

$$z(k) = f(I(k)). \quad (2)$$

This says the displacement at time step  $k$  depends only on the current at the same time step. That is too limited for actuator dynamics. A soft actuator has delayed response, damping, inertia, and possible memory effects. The same current value can produce different displacement values depending on whether the current is rising, falling, or has been held for some time.

A dynamic model should instead use

$$z(k) = f(I(1), I(2), \dots, I(k)). \quad (3)$$

This means that all previous current values can affect the current response. An LSTM is designed for this type of sequence modeling.

## 4 What Is an LSTM at a High Level?

LSTM stands for Long Short Term Memory. It is a recurrent neural network designed for sequence data. Sequence data means data ordered in time, such as actuator current, displacement, and force.

The most important idea is memory. An LSTM has internal variables that carry information from previous time steps to later time steps. This makes it useful for dynamic systems. In this actuator problem, the LSTM can remember how the current has changed over time and use that memory to predict the current displacement and force.

At a high level, the LSTM receives the current sequence one time step at a time:

$$I(1), I(2), I(3), \dots, I(k).$$

At every time step, it updates its internal memory and outputs predicted actuator responses:

$$\hat{z}(k), \quad \hat{F}_c(k).$$

## 5 Step 1: Reading the Data

The MATLAB script reads three sheets from the COMSOL Excel file:

- Displacement sheet.
- Force sheet.
- Current sheet.

The script renames the variables to clean names:

```
Time_s, Displacement_mm, CoilForce_N, WeightLoad_N, Current_A.
```

This makes the rest of the code easier to read and avoids problems caused by long or inconsistent column names in the Excel file.

## 6 Step 2: Cleaning Duplicate Time Stamps

COMSOL exports can include repeated time values. If two or more rows have the same time value, interpolation and training can become unreliable. The script uses a helper function called `cleanTimeTable`. This function:

1. Removes missing rows.
2. Sorts the table by time.
3. Finds repeated time values.
4. Averages the signal values that belong to the same time.

After this step, each time value appears only once.

## 7 Step 3: Synchronizing the Signals

The force, displacement, and current data may not be listed on exactly the same time grid. The code uses the force time vector as the common time grid:

$$t = t_F.$$

Then it interpolates current and displacement onto that time grid:

$$I_{\text{grid}} = \text{interp1}(t_I, I, t_F), \quad (4)$$

$$z_{\text{grid}} = \text{interp1}(t_z, z, t_F). \quad (5)$$

This gives synchronized variables:

$$t, \quad I(t), \quad z(t), \quad F_c(t), \quad F_w(t).$$

The net force is also computed:

$$F_{\text{net}}(t) = F_c(t) - F_w(t). \quad (6)$$

## 8 Step 4: Plotting the Data Before Training

Before training the LSTM, the script plots the physical signals one by one. This is important because the data must make physical sense before being used for machine learning. The plots include:

- Current versus time.
- Coil force versus time.
- Weight or load force versus time.
- Net force versus time.
- Displacement versus time.
- Synchronized current, force, and displacement.

- Early trigger response from 0 to 30 ms.
- Force versus current.
- Displacement versus current.
- Force versus displacement.

The early trigger plot is especially useful because it checks that the response happens over a realistic time window rather than appearing as an instantaneous jump.

## 9 Step 5: Defining the LSTM Input and Output

The input sequence is current:

$$X(k) = I(k). \quad (7)$$

The output sequence contains displacement and coil force:

$$Y(k) = \begin{bmatrix} z(k) \\ F_c(k) \end{bmatrix}. \quad (8)$$

In MATLAB sequence format, the input and output are arranged as:

$$X \in \mathbb{R}^{1 \times T}, \quad Y \in \mathbb{R}^{2 \times T}.$$

The number 1 means there is one input feature. The number 2 means there are two output features. The variable  $T$  is the number of time samples.

## 10 Step 6: Splitting Training and Testing Data

The code uses the first 75 percent of the time history for training:

$$\text{training data} = 0\% \text{ to } 75\% \text{ of the sequence.}$$

The final 25 percent is used for testing:

$$\text{testing data} = 75\% \text{ to } 100\% \text{ of the sequence.}$$

This preserves time order. The code does not shuffle the sequence because shuffling would destroy the dynamic meaning of the data.

## 11 Step 7: Normalization

The code normalizes the input and output data before training. Normalization means subtracting the mean and dividing by the standard deviation:

$$\tilde{X} = \frac{X - \mu_X}{\sigma_X}, \quad (9)$$

$$\tilde{Y} = \frac{Y - \mu_Y}{\sigma_Y}. \quad (10)$$

This is necessary because the variables have different units and scales:

- Current is in amperes.
- Displacement is in millimeters.
- Force is in Newtons.

Without normalization, the network may focus more on the variable with the largest numerical magnitude and less on the smaller scale variables.

A very important detail is that the mean and standard deviation are computed from the training data only. This prevents information from the test data from leaking into the training process.

## 12 Step 8: Network Architecture

The network architecture is:

sequence input  $\rightarrow$  LSTM  $\rightarrow$  fully connected  $\rightarrow$  ReLU  $\rightarrow$  fully connected  $\rightarrow$  regression.

### 12.1 Sequence Input Layer

The sequence input layer tells MATLAB that the model receives a time sequence. Since the only input is current, the input size is one.

### 12.2 LSTM Layer

The LSTM layer has 64 hidden units. These hidden units are the internal memory variables of the network. They allow the model to remember previous input information and use it to predict current outputs.

### 12.3 Fully Connected Layer

The first fully connected layer has 32 units. It transforms the LSTM memory representation into a feature representation that is useful for output prediction.

## 12.4 ReLU Layer

The ReLU layer adds nonlinearity:

$$\text{ReLU}(x) = \max(0, x).$$

This allows the network to learn nonlinear actuator behavior.

## 12.5 Final Fully Connected Layer

The final fully connected layer has two outputs:

$$\hat{z}(k), \quad \hat{F}_c(k).$$

These are the predicted displacement and predicted coil force.

## 12.6 Regression Layer

The regression layer tells MATLAB that this is a continuous output prediction problem. The model is not classifying categories. It is predicting numerical physical quantities.

# 13 Step 9: Training Options

The model uses the Adam optimizer with a learning rate of  $10^{-3}$ . Adam is a common optimizer for neural network training because it adapts the update size during training. The model is trained for 600 epochs. One epoch means one full pass through the training sequence.

The mini batch size is 1 because the code uses one full time sequence. The shuffle option is set to never because time order is important.

# 14 Step 10: Prediction

After training, the model predicts the entire time history. The output is first produced in normalized units. Then the code converts it back to physical units:

$$\hat{Y} = \tilde{Y}\sigma_Y + \mu_Y. \tag{11}$$

This gives predictions in millimeters and Newtons.

# 15 Step 11: Error Calculation

The displacement error is:

$$e_z(k) = z(k) - \hat{z}(k). \tag{12}$$

The coil force error is:

$$e_F(k) = F_c(k) - \hat{F}_c(k). \tag{13}$$

The root mean square error is:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N e(k)^2}. \quad (14)$$

The mean absolute error is:

$$\text{MAE} = \frac{1}{N} \sum_{k=1}^N |e(k)|. \quad (15)$$

RMSE is sensitive to larger errors. MAE gives the average absolute error.

## 16 Step 12: Results

The run produced the following reported accuracy:

Table 1: LSTM performance metrics from the MATLAB run.

| Output       | Training RMSE              | Testing RMSE               |
|--------------|----------------------------|----------------------------|
| Displacement | 0.0370165 mm               | 0.00503445 mm              |
| Coil force   | $7.43668 \times 10^{-4}$ N | $8.65539 \times 10^{-5}$ N |

These results show that the model can reproduce the available single actuator trajectory accurately. However, this does not yet prove that the model will generalize to new current profiles. To prove generalization, the model should be trained on multiple current excitations.

## 17 Step 13: Saved Outputs

The script saves:

- Cleaned synchronized data as `.csv` and `.mat`.
- The trained LSTM model as `ESA_single_actuator_LSTM_model.mat`.
- Prediction results as `ESA_single_actuator_LSTM_results.csv`.
- A text summary as `ESA_single_actuator_LSTM_summary.txt`.
- All plots in the `figures` folder.

## 18 How This Connects to the Larger Actuator Network Work

The older model represents a network of actuators using a state space formulation. That model is physics based and includes many states and inputs. The present LSTM model is only for one actuator. Therefore, the correct relationship is:

$$\text{single actuator LSTM} \neq \text{full actuator network model}.$$

Instead, the single actuator LSTM is a building block. Later, several single actuator data driven models can be combined or extended to form a larger data driven model of an actuator array.

## 19 What Should Be Done Next

The next step is to generate or collect more data. The most useful future datasets would include:

- Different current amplitudes.
- Different ramp times.
- Pulse inputs.
- Step inputs.
- Different load conditions.
- Temperature dependent cases.
- Multiple actuator configurations.

With multiple sequences, the training data would become:

$$\{X_1, X_2, \dots, X_M\},$$

where each  $X_i$  is a different current excitation case. This would allow the LSTM to learn a more general actuator model rather than only one response trajectory.

## 20 Conclusion

The MATLAB simulation trains a first LSTM dynamic model for one electromagnetic soft actuator. The model uses current history to predict displacement and coil force. The key idea is that the LSTM provides a memory based data driven approximation of actuator dynamics. The present results are promising for the available COMSOL trajectory, but broader training data are required before the model can be used as a general predictive model.